Universidad CENFOTEC

Escuela de Ingeniería del Software

Proyecto: Misión Órbita

Primer avance: Preparación de la misión

Código del curso: SOFT-01
Nombre del curso: Principios de programación 1
Sección: SCV6
Periodo: C3-2026
Docente facilitador: Verónica Mora Lezcano
Fecha de entrega: 14 de octubre de 2026

Integrante

Grace Hernandez Guevara

Contenido del documento

Este documento presenta el primer avance del proyecto Misión Órbita: cuatro programas independientes en Python que representan la preparación de la misión. Para cada ejercicio se incluyen la tabla de variables, el diagrama de flujo, el código fuente y la tabla de casos de prueba con su evidencia de ejecución.

Ejercicio	Descripción	Estructura de control	Archivo
Ejercicio 1	Cálculo de recursos necesarios para una misión	Secuencial	`ejercicio1_recursos.py`
Ejercicio 2	Verificación de combustible	Condicional simple	`ejercicio2_combustible.py`
Ejercicio 3	Verificación del soporte para la tripulación	Condicional doble	`ejercicio3_soporte_tripulacion.py`
Ejercicio 4	Autorización inicial de lanzamiento	Condicional doble sin anidación	`ejercicio4_autorizacion.py`

Estructura del repositorio

```
mision-orbita/
├── README.md
├── avance1/
│   ├── ejercicio1_recursos.py
│   ├── ejercicio2_combustible.py
│   ├── ejercicio3_soporte_tripulacion.py
│   └── ejercicio4_autorizacion.py
└── documentacion/
    └── avance1.pdf
```

Ejercicio 1: Cálculo de recursos necesarios para una misión

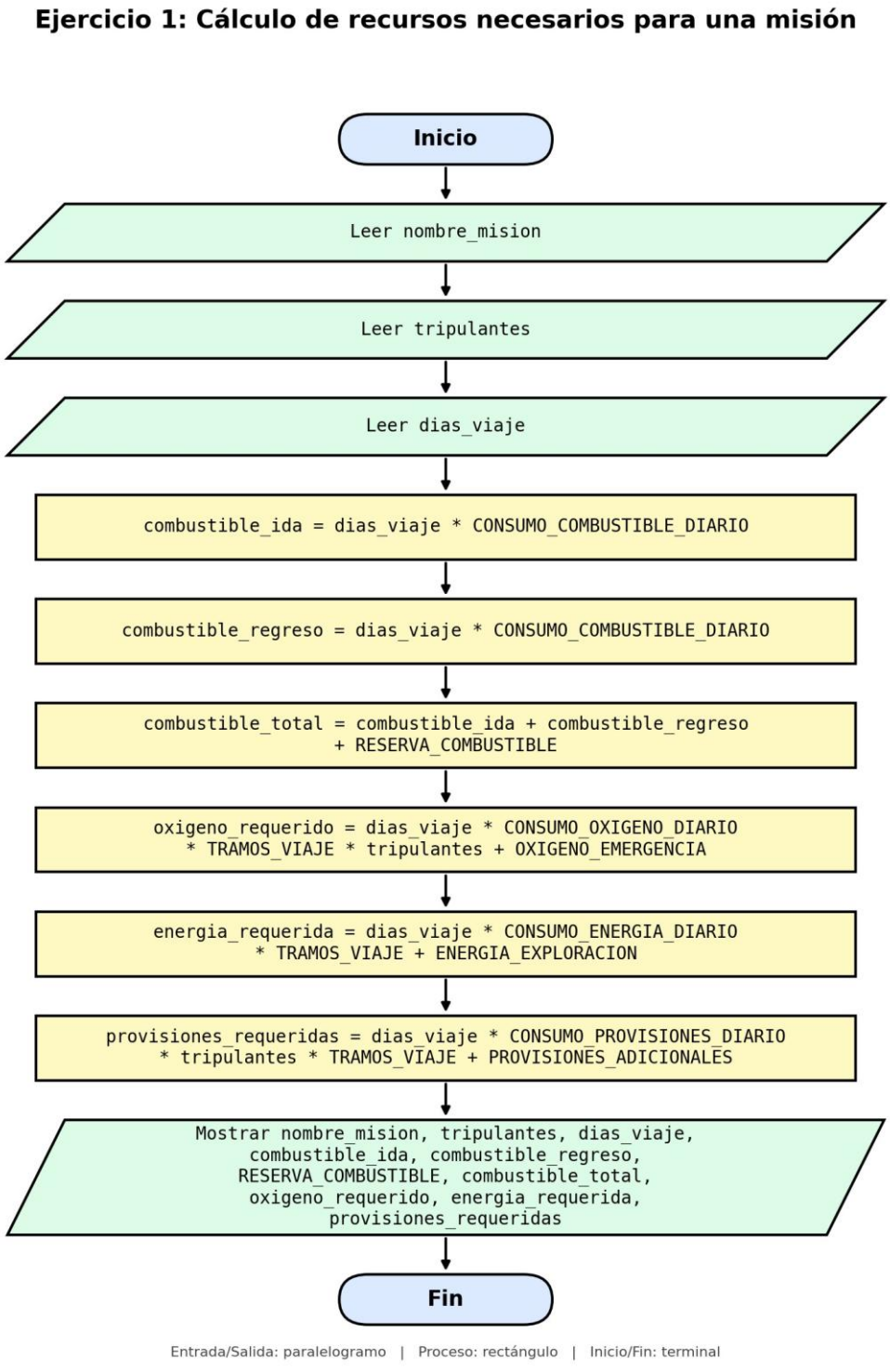
Calcula los recursos que requiere una misión (combustible de ida, de regreso, reserva y total, oxígeno, energía y provisiones).
Estructura de control: secuencial.

1. Tabla de variables

Nombre	Tipo de dato	Clasificación	Descripción	Valor inicial / restricción
CONSUMO_COMBUSTIBLE_DIARIO	int	Constante	Combustible consumido por día de viaje	8
RESERVA_COMBUSTIBLE	int	Constante	Combustible adicional de emergencia	10
TRAMOS_VIAJE	int	Constante	Tramos del viaje (ida y regreso)	2
CONSUMO_OXIGENO_DIARIO	int	Constante	Oxígeno por tripulante por día	2
OXIGENO_EMERGENCIA	int	Constante	Oxígeno adicional de emergencia	5
CONSUMO_ENERGIA_DIARIO	int	Constante	Energía consumida por día	5
ENERGIA_EXPLORACION	int	Constante	Energía para operaciones de exploración	10
CONSUMO_PROVISIONES_DIARIO	int	Constante	Provisiones por tripulante por día	1
PROVISIONES_ADICIONALES	int	Constante	Provisiones adicionales	3
nombre_mision	str	Entrada	Nombre de la misión	Texto
tripulantes	int	Entrada	Cantidad de tripulantes	Entero positivo
dias_viaje	int	Entrada	Días estimados para llegar al destino	Entero positivo
combustible_ida	int	Intermedia	Combustible para llegar	dias_viaje * CONSUMO_COMBUSTIBLE_DIARIO
combustible_regreso	int	Intermedia	Combustible para regresar	dias_viaje * CONSUMO_COMBUSTIBLE_DIARIO
combustible_total	int	Salida	Combustible total requerido	combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE
oxigeno_requerido	int	Salida	Oxígeno requerido para la misión	dias_viaje * CONSUMO_OXIGENO_DIARIO * TRAMOS_VIAJE * tripulantes + OXIGENO_EMERGENCIA
energia_requerida	int	Salida	Energía requerida para la misión	dias_viaje * CONSUMO_ENERGIA_DIARIO * TRAMOS_VIAJE + ENERGIA_EXPLORACION
provisiones_requeridas	int	Salida	Provisiones requeridas para la misión	dias_viaje * CONSUMO_PROVISIONES_DIARIO * tripulantes * TRAMOS_VIAJE + PROVISIONES_ADICIONALES

Ejercicio 1: Cálculo de recursos necesarios para una misión

2. Diagrama de flujo



Ejercicio 1: Cálculo de recursos necesarios para una misión

3. Código fuente (ejercicio1_recursos.py)

```
# ============================================================
# Archivo: ejercicio1_recursos.py
```

```
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV6
# Integrantes: Grace Hernandez Guevara
# Fecha: 14/10/2026     Versión: 1.0 # Descripción: Calcula los
recursos necesarios para una misión.
# ============================================================

# --- Declaración de constantes --# Se declaran al inicio para evitar
números mágicos en las fórmulas.
CONSUMO_COMBUSTIBLE_DIARIO = 8
RESERVA_COMBUSTIBLE = 10
TRAMOS_VIAJE = 2  # El viaje tiene ida y regreso con la misma duración
CONSUMO_OXIGENO_DIARIO = 2
OXIGENO_EMERGENCIA = 5
CONSUMO_ENERGIA_DIARIO = 5
ENERGIA_EXPLORACION = 10
CONSUMO_PROVISIONES_DIARIO = 1
PROVISIONES_ADICIONALES = 3

# --- Entrada de datos --nombre_mision = input("Nombre de la
misión: ") tripulantes = int(input("Cantidad de tripulantes: "))
dias_viaje = int(input("Días estimados para llegar al destino: "))

# --- Procesamiento --# El regreso dura lo mismo que la ida, por eso usa el mismo
consumo diario. combustible_ida = dias_viaje * CONSUMO_COMBUSTIBLE_DIARIO
combustible_regreso = dias_viaje * CONSUMO_COMBUSTIBLE_DIARIO combustible_total =
combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE

# El oxígeno, la energía y las provisiones se consumen en ida y regreso. oxigeno_requerido = dias_viaje *
CONSUMO_OXIGENO_DIARIO * TRAMOS_VIAJE * tripulantes + OXIGENO_EMERGENCIA energia_requerida = dias_viaje *
CONSUMO_ENERGIA_DIARIO * TRAMOS_VIAJE + ENERGIA_EXPLORACION provisiones_requeridas = dias_viaje *
CONSUMO_PROVISIONES_DIARIO * tripulantes * TRAMOS_VIAJE + PROVISIONES_ADICIONALES

# --- Salida de resultados --print("Misión:", nombre_mision)
print("Tripulantes:", tripulantes) print("Duración estimada hasta el
destino:", dias_viaje, "días") print("Combustible para llegar:",
combustible_ida, "unidades") print("Combustible para regresar:",
combustible_regreso, "unidades") print("Reserva de combustible:",
RESERVA_COMBUSTIBLE, "unidades") print("Combustible total
requerido:", combustible_total, "unidades") print("Oxígeno
requerido:", oxigeno_requerido, "unidades") print("Energía
requerida:", energia_requerida, "unidades") print("Provisiones
requeridas:", provisiones_requeridas, "unidades")
```

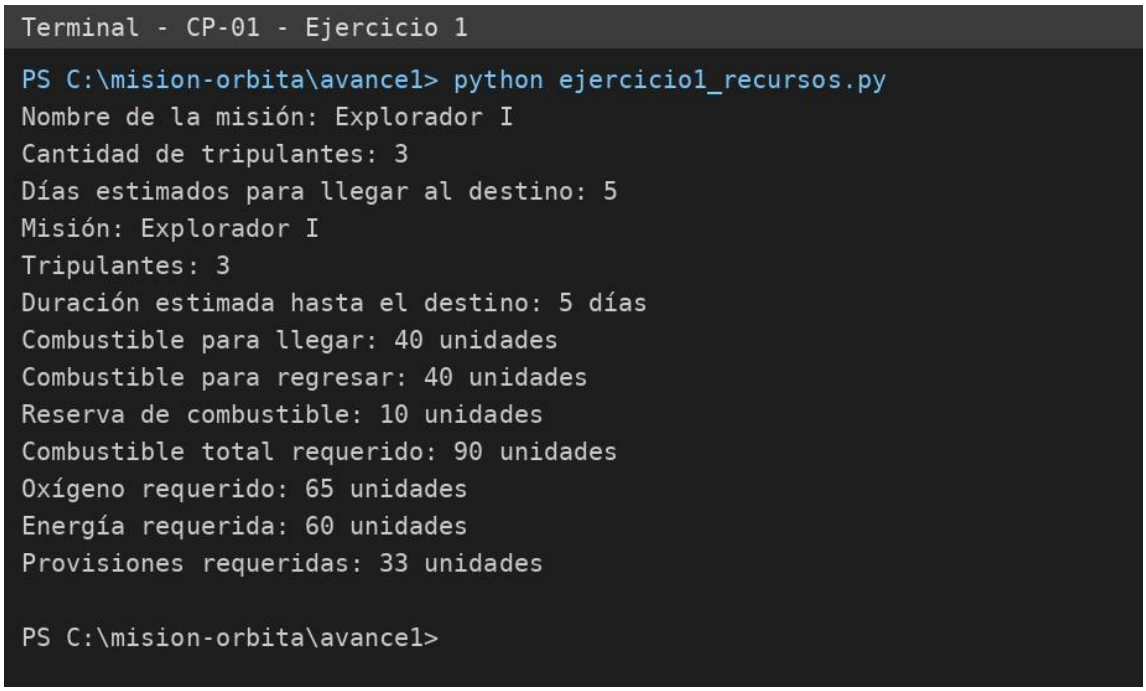
4. Casos de prueba

ID	Ejercicio	Escenario	Datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-01	Ejercicio 1	Ejemplo de la consigna	Misión: Explorador I Tripulantes: 3 Días: 5	Ida 40, regreso 40, reserva 10, total 90, oxígeno 65, energía 60, provisiones 33	Igual al esperado	Exitoso
CP-02	Ejercicio 1	Valores mínimos (1 tripulante, 1 día)	Misión: Mini Tripulantes: 1 Días: 1	Ida 8, regreso 8, reserva 10, total 26, oxígeno 9, energía 20, provisiones 5	Igual al esperado	Exitoso
CP-03	Ejercicio 1	Misión larga con más tripulación	Misión: Artemis II Tripulantes: 4 Días: 10	Ida 80, regreso 80, reserva 10, total 170, oxígeno 165, energía 110, provisiones 83	Igual al esperado	Exitoso

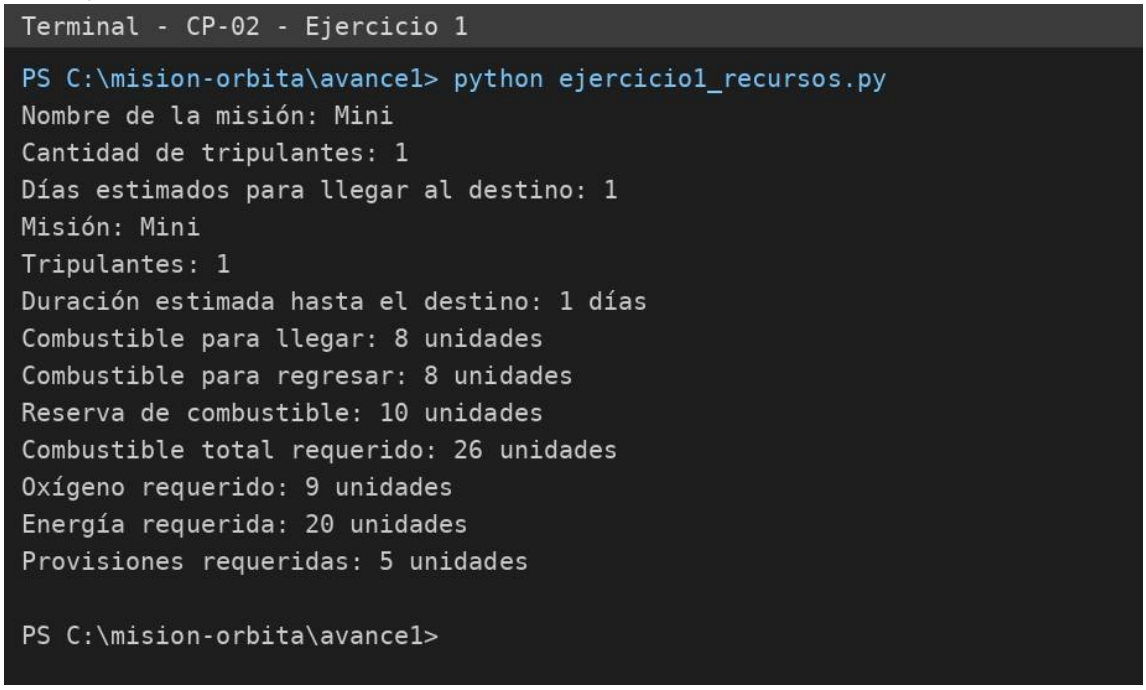
Los resultados esperados se calcularon manualmente antes de ejecutar el programa.

5. Evidencia de ejecución

Captura CP-01 - Ejercicio 1



Captura CP-02 - Ejercicio 1



Captura CP-03 - Ejercicio 1

```
Terminal - CP-03 - Ejercicio 1
PS C:\mision-orbita\avance1> python ejercicio1_recursos.py
Nombre de la misión: Artemis II
Cantidad de tripulantes: 4
Días estimados para llegar al destino: 10
Misión: Artemis II
Tripulantes: 4
Duración estimada hasta el destino: 10 días
Combustible para llegar: 80 unidades
Combustible para regresar: 80 unidades
Reserva de combustible: 10 unidades
Combustible total requerido: 170 unidades
Oxígeno requerido: 165 unidades
Energía requerida: 110 unidades
Provisiones requeridas: 83 unidades

PS C:\mision-orbita\avance1>
```

Ejercicio 2: Verificación de combustible

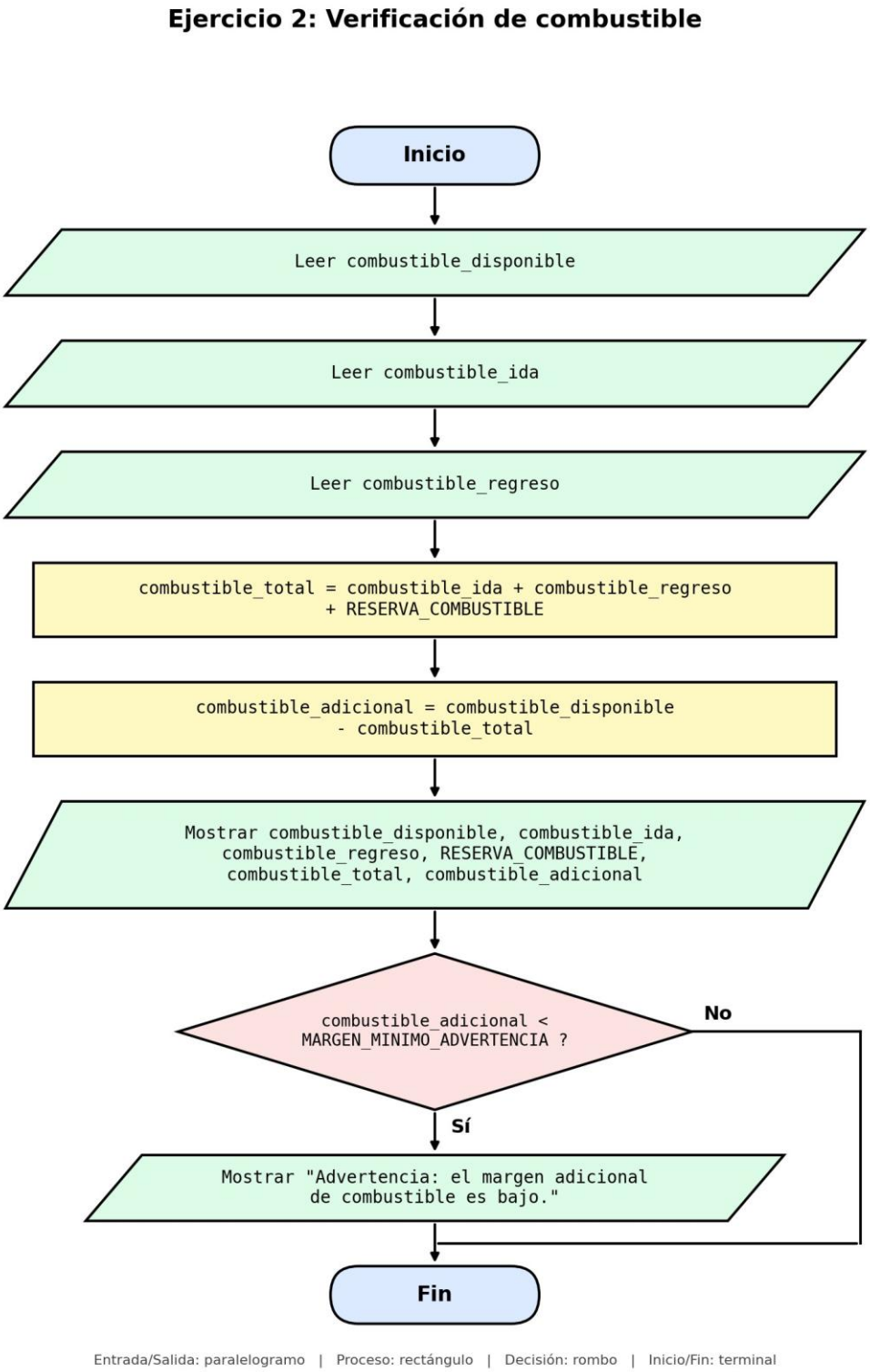
Verifica si la nave tiene combustible suficiente y muestra una advertencia cuando el margen adicional es menor que 10 unidades. Estructura de control: condicional simple.

1. Tabla de variables

Nombre	Tipo de dato	Clasificación	Descripción	Valor inicial / restricción
`RESERVA_COMBUSTIBLE`	int	Constante	Reserva de emergencia que se suma al total	10
`MARGEN_MINIMO_ADVERTENCIA`	int	Constante	Margen adicional mínimo antes de advertir	10
`combustible_disponible`	int	Entrada	Combustible con que cuenta la nave	Entero positivo
`combustible_ida`	int	Entrada	Combustible requerido para el viaje de ida	Entero positivo
`combustible_regreso`	int	Entrada	Combustible requerido para el viaje de regreso	Entero positivo
`combustible_total`	int	Salida	Total requerido (ida + regreso + reserva)	`combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE`
`combustible_adicional`	int	Salida	Diferencia entre lo disponible y el total requerido (puede ser negativa)	`combustible_disponible combustible_total`

Ejercicio 2: Verificación de combustible

2. Diagrama de flujo



Ejercicio 2: Verificación de combustible

3. Código fuente (ejercicio2_combustible.py)

```
# ============================================================
# Archivo: ejercicio2_combustible.py
```

```
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV6
# Integrantes: Grace Hernandez Guevara
# Fecha: 14/10/2026     Versión: 1.0
# Descripción: Verifica si una nave posee suficiente combustible
#              y advierte cuando el margen adicional es bajo.
# ============================================================

# --- Declaración de constantes --# Se declaran al inicio para
evitar números mágicos en las fórmulas.
RESERVA_COMBUSTIBLE = 10
MARGEN_MINIMO_ADVERTENCIA = 10

# --- Entrada de datos --combustible_disponible = int(input("Combustible disponible:
")) combustible_ida = int(input("Combustible requerido para el viaje de ida: "))
combustible_regreso = int(input("Combustible requerido para el viaje de regreso: "))

# --- Procesamiento --# El total incluye el regreso y la reserva: no basta con
llegar al destino. combustible_total = combustible_ida + combustible_regreso +
RESERVA_COMBUSTIBLE combustible_adicional = combustible_disponible -
combustible_total

# --- Salida de resultados --# Estas salidas se muestran siempre, sin importar
el margen adicional. print("Combustible disponible:", combustible_disponible,
"unidades") print("Combustible requerido para llegar:", combustible_ida,
"unidades") print("Combustible requerido para regresar:", combustible_regreso,
"unidades") print("Reserva de emergencia:", RESERVA_COMBUSTIBLE, "unidades")
print("Combustible total requerido:", combustible_total, "unidades")
print("Combustible adicional:", combustible_adicional, "unidades")

# --- Decisión --# Condicional simple: la advertencia solo aparece si el margen
es menor que 10. # Se usa "<" y no "<=" porque con exactamente 10 unidades el
margen es aceptable. if combustible_adicional < MARGEN_MINIMO_ADVERTENCIA:
print("Advertencia: el margen adicional de combustible es bajo.")
```

4. Casos de prueba

ID	Ejercicio	Escenario	Datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-04	Ejercicio 2	Recursos suficientes	Disponible = 150 Ida = 40 Regreso = 40	Total = 90 Adicional = 60 No muestra advertencia	Igual al esperado	Exitoso
CP-05	Ejercicio 2	Exactamente el mínimo requerido	Disponible = 90 Ida = 40 Regreso = 40	Total = 90 Adicional = 0 Muestra advertencia	Igual al esperado	Exitoso
CP-06	Ejercicio 2	Margen adicional bajo (ejemplo de la consigna)	Disponible = 95 Ida = 40 Regreso = 40	Total = 90 Adicional = 5 Muestra advertencia	Igual al esperado	Exitoso
CP-07	Ejercicio 2	Combustible suficiente solo para llegar	Disponible = 45 Ida = 40 Regreso = 40	Total = 90 Adicional = -45 Muestra advertencia	Igual al esperado	Exitoso
CP-08	Ejercicio 2	Límite de la advertencia (adicional = 10)	Disponible = 100 Ida = 40 Regreso = 40	Total = 90 Adicional = 10 No muestra advertencia	Igual al esperado	Exitoso
ID	**Ejercicio**	**Escenario**	**Datos de entrada**	**Resultado esperado**	**Resultado obtenido**	**Estado**
CP-09	Ejercicio 2	Recursos insuficientes	Disponible = 20 Ida = 40 Regreso = 40	Total = 90 Adicional = -70 Muestra advertencia	Igual al esperado	Exitoso

Los resultados esperados se calcularon manualmente antes de ejecutar el programa.

5. Evidencia de ejecución

Captura CP-04 - Ejercicio 2

```
Terminal - CP-04 - Ejercicio 2
PS C:\mision-orbita\avance1> python ejercicio2_combustible.py
Combustible disponible: 150
Combustible requerido para el viaje de ida: 40
Combustible requerido para el viaje de regreso: 40
Combustible disponible: 150 unidades
Combustible requerido para llegar: 40 unidades
Combustible requerido para regresar: 40 unidades
Reserva de emergencia: 10 unidades
Combustible total requerido: 90 unidades
Combustible adicional: 60 unidades

PS C:\mision-orbita\avance1>
```

Captura CP-05 - Ejercicio 2

```
Terminal - CP-05 - Ejercicio 2
PS C:\mision-orbita\avance1> python ejercicio2_combustible.py
Combustible disponible: 90
Combustible requerido para el viaje de ida: 40
Combustible requerido para el viaje de regreso: 40
Combustible disponible: 90 unidades
Combustible requerido para llegar: 40 unidades
Combustible requerido para regresar: 40 unidades
Reserva de emergencia: 10 unidades
Combustible total requerido: 90 unidades
Combustible adicional: 0 unidades
Advertencia: el margen adicional de combustible es bajo.

PS C:\mision-orbita\avance1>
```

Captura CP-06 - Ejercicio 2

```
Terminal - CP-06 - Ejercicio 2
PS C:\mision-orbita\avance1> python ejercicio2_combustible.py
Combustible disponible: 95
Combustible requerido para el viaje de ida: 40
Combustible requerido para el viaje de regreso: 40
Combustible disponible: 95 unidades
Combustible requerido para llegar: 40 unidades
Combustible requerido para regresar: 40 unidades
Reserva de emergencia: 10 unidades
Combustible total requerido: 90 unidades
Combustible adicional: 5 unidades
Advertencia: el margen adicional de combustible es bajo.

PS C:\mision-orbita\avance1>
```

Captura CP-07 - Ejercicio 2

```
Terminal - CP-07 - Ejercicio 2
PS C:\mision-orbita\avance1> python ejercicio2_combustible.py
Combustible disponible: 45
Combustible requerido para el viaje de ida: 40
Combustible requerido para el viaje de regreso: 40
Combustible disponible: 45 unidades
Combustible requerido para llegar: 40 unidades
Combustible requerido para regresar: 40 unidades
Reserva de emergencia: 10 unidades
Combustible total requerido: 90 unidades
Combustible adicional: -45 unidades
Advertencia: el margen adicional de combustible es bajo.

PS C:\mision-orbita\avance1>
```

Captura CP-08 - Ejercicio 2

```
Terminal - CP-08 - Ejercicio 2
PS C:\mision-orbita\avance1> python ejercicio2_combustible.py
Combustible disponible: 100
Combustible requerido para el viaje de ida: 40
Combustible requerido para el viaje de regreso: 40
Combustible disponible: 100 unidades
Combustible requerido para llegar: 40 unidades
Combustible requerido para regresar: 40 unidades
Reserva de emergencia: 10 unidades
Combustible total requerido: 90 unidades
Combustible adicional: 10 unidades

PS C:\mision-orbita\avance1>
```

Captura CP-09 - Ejercicio 2

```
Terminal - CP-09 - Ejercicio 2
PS C:\mision-orbita\avance1> python ejercicio2_combustible.py
Combustible disponible: 20
Combustible requerido para el viaje de ida: 40
Combustible requerido para el viaje de regreso: 40
Combustible disponible: 20 unidades
Combustible requerido para llegar: 40 unidades
Combustible requerido para regresar: 40 unidades
Reserva de emergencia: 10 unidades
Combustible total requerido: 90 unidades
Combustible adicional: -70 unidades
Advertencia: el margen adicional de combustible es bajo.

PS C:\mision-orbita\avance1>
```

Ejercicio 3: Verificación del soporte para la tripulación

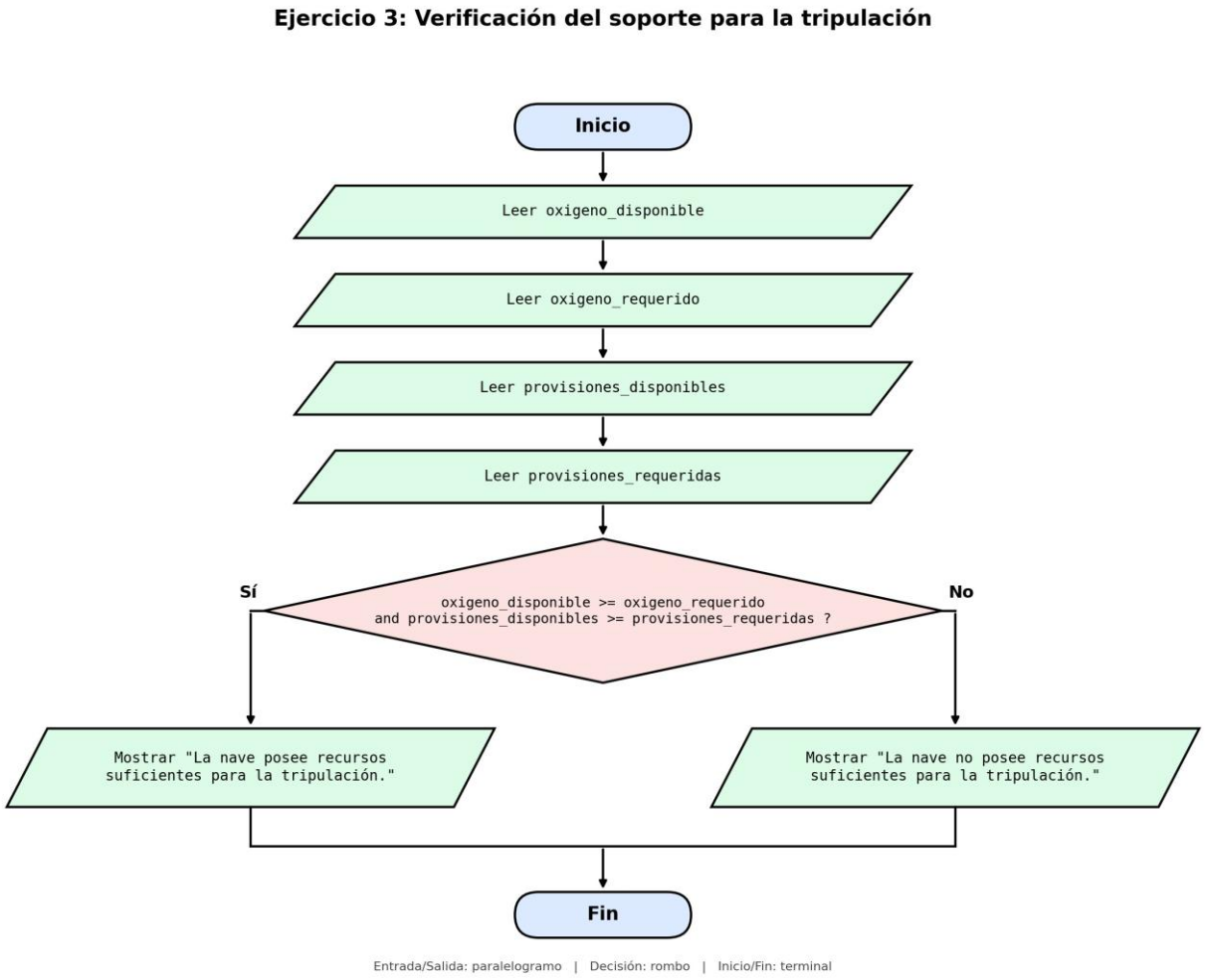
Verifica si el oxígeno y las provisiones disponibles alcanzan para la tripulación, sin indicar cuál recurso falla. Estructura de control: condicional doble (if/else).

1. Tabla de variables

Nombre	Tipo de dato	Clasificación	Descripción	Valor inicial / restricción
`oxigeno_disponible`	int	Entrada	Oxígeno con que cuenta la nave	Entero positivo
`oxigeno_requerido`	int	Entrada	Oxígeno necesario para la tripulación	Entero positivo
`provisiones_disponibles`	int	Entrada	Provisiones con que cuenta la nave	Entero positivo
`provisiones_requeridas`	int	Entrada	Provisiones necesarias para la tripulación	Entero positivo

Ejercicio 3: Verificación del soporte para la tripulación

2. Diagrama de flujo



Ejercicio 3: Verificación del soporte para la tripulación

3. Código fuente (ejercicio3_soporte_tripulacion.py)

```
# ============================================================
# Archivo: ejercicio3_soporte_tripulacion.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV6
# Integrantes: Grace Hernandez Guevara
# Fecha: 14/10/2026     Versión: 1.0
# Descripción: Verifica si la nave posee oxígeno y provisiones
#              suficientes para sostener a toda la tripulación.
# ============================================================

# --- Entrada de datos --oxigeno_disponible = int(input("Oxígeno
disponible: ")) oxigeno_requerido = int(input("Oxígeno requerido:
")) provisiones_disponibles = int(input("Provisiones disponibles:
")) provisiones_requeridas = int(input("Provisiones requeridas:
"))

# --- Decisión y salida de resultados ---
# Se usa "and" porque el soporte solo es adecuado si AMBOS recursos alcanzan; # se usa ">="
porque tener exactamente la cantidad requerida es suficiente. # No se indica cuál recurso falla,
tal como lo pide la consigna. if oxigeno_disponible >= oxigeno_requerido and
provisiones_disponibles >= provisiones_requeridas:
    print("La nave posee recursos suficientes para la tripulación.")
else:     print("La nave no posee recursos suficientes para la
tripulación.")
```

4. Casos de prueba

ID	Ejercicio	Escenario	Datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-10	Ejercicio 3	Ambos recursos suficientes	O. disp. = 100 O. req. = 65 Prov. disp. = 50 Prov. req. = 33	La nave posee recursos suficientes para la tripulación.	Igual al esperado	Exitoso
CP-11	Ejercicio 3	Ambos exactamente iguales al mínimo (límite)	O. disp. = 65 O. req. = 65 Prov. disp. = 33 Prov. req. = 33	La nave posee recursos suficientes para la tripulación.	Igual al esperado	Exitoso
CP-12	Ejercicio 3	Oxígeno insuficiente, provisiones suficientes	O. disp. = 64 O. req. = 65 Prov. disp. = 50 Prov. req. = 33	La nave no posee recursos suficientes para la tripulación.	Igual al esperado	Exitoso
CP-13	Ejercicio 3	Provisiones insuficientes, oxígeno suficiente	O. disp. = 100 O. req. = 65 Prov. disp. = 32 Prov. req. = 33	La nave no posee recursos suficientes para la tripulación.	Igual al esperado	Exitoso
CP-14	Ejercicio 3	Ambos recursos insuficientes	O. disp. = 10 O. req. = 65 Prov. disp. = 5 Prov. req. = 33	La nave no posee recursos suficientes para la tripulación.	Igual al esperado	Exitoso

Los resultados esperados se calcularon manualmente antes de ejecutar el programa.

5. Evidencia de ejecución

Captura CP-10 - Ejercicio 3

```
Terminal - CP-10 - Ejercicio 3
PS C:\mision-orbita\avance1> python ejercicio3_soporte_tripulacion.py
Oxígeno disponible: 100
Oxígeno requerido: 65
Provisiones disponibles: 50
Provisiones requeridas: 33
La nave posee recursos suficientes para la tripulación.

PS C:\mision-orbita\avance1>
```

Captura CP-11 - Ejercicio 3

```
Terminal - CP-11 - Ejercicio 3
PS C:\mision-orbita\avance1> python ejercicio3_soporte_tripulacion.py
Oxígeno disponible: 65
Oxígeno requerido: 65
Provisiones disponibles: 33
Provisiones requeridas: 33
La nave posee recursos suficientes para la tripulación.

PS C:\mision-orbita\avance1>
```

Captura CP-12 - Ejercicio 3

```
Terminal - CP-12 - Ejercicio 3
PS C:\mision-orbita\avance1> python ejercicio3_soporte_tripulacion.py
Oxígeno disponible: 64
Oxígeno requerido: 65
Provisiones disponibles: 50
Provisiones requeridas: 33
La nave no posee recursos suficientes para la tripulación.

PS C:\mision-orbita\avance1>
```

Captura CP-13 - Ejercicio 3

```
Terminal - CP-13 - Ejercicio 3
PS C:\mision-orbita\avance1> python ejercicio3_soporte_tripulacion.py
Oxígeno disponible: 100
Oxígeno requerido: 65
Provisiones disponibles: 32
Provisiones requeridas: 33
La nave no posee recursos suficientes para la tripulación.

PS C:\mision-orbita\avance1>
```

Captura CP-14 - Ejercicio 3

```
Terminal - CP-14 - Ejercicio 3
PS C:\mision-orbita\avance1> python ejercicio3_soporte_tripulacion.py
Oxígeno disponible: 10
Oxígeno requerido: 65
Provisiones disponibles: 5
Provisiones requeridas: 33
La nave no posee recursos suficientes para la tripulación.

PS C:\mision-orbita\avance1>
```

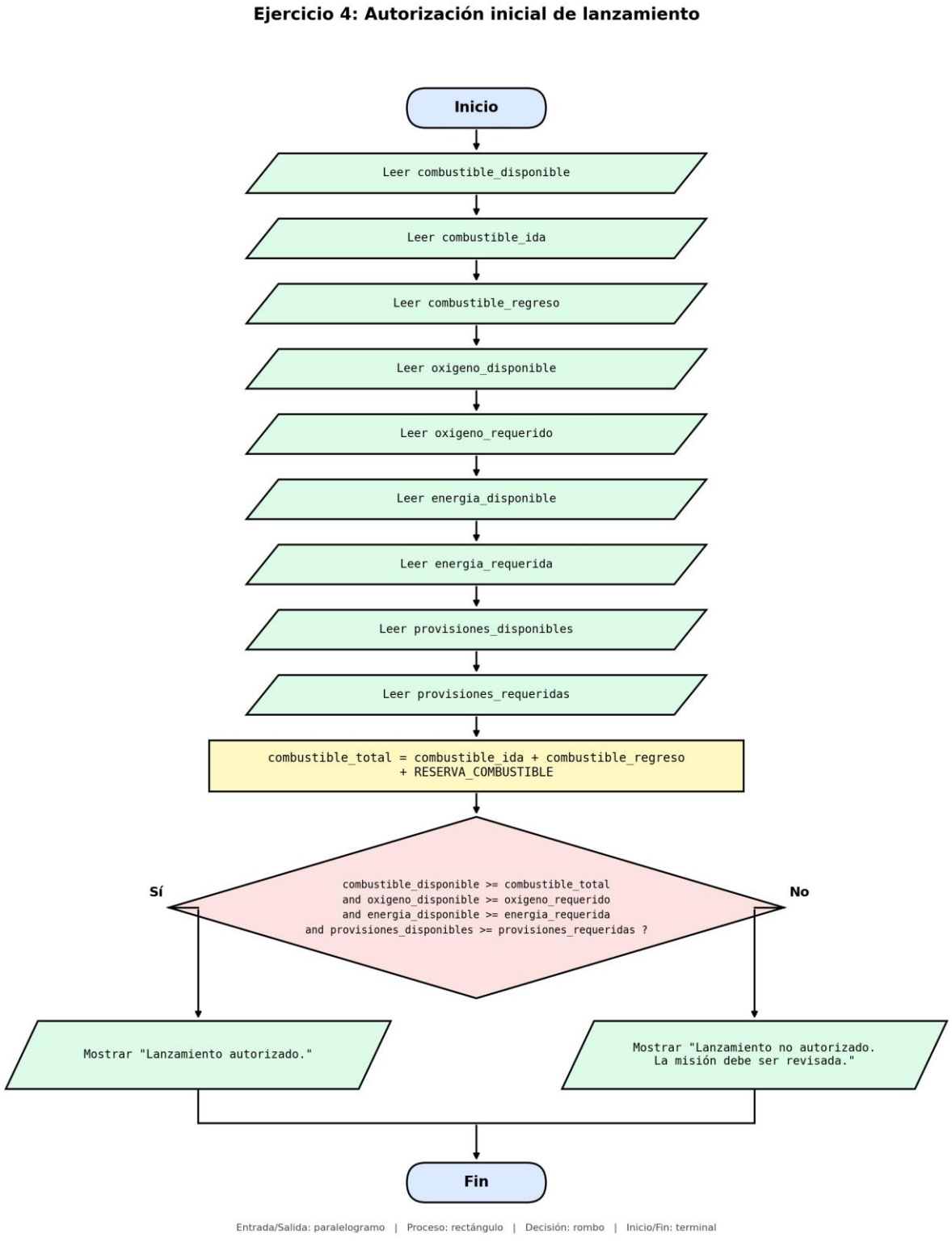
Ejercicio 4: Autorización inicial de lanzamiento

Autoriza o rechaza el lanzamiento evaluando combustible (contra el total con regreso y reserva), oxígeno, energía y provisiones. Estructura de control: condicional doble sin anidación.

1. Tabla de variables

Nombre	Tipo de dato	Clasificación	Descripción	Valor inicial / restricción
`RESERVA_COMBUSTIBLE`	int	Constante	Reserva de emergencia que se suma al combustible total	10
`combustible_disponible`	int	Entrada	Combustible con que cuenta la nave	Entero positivo
`combustible_ida`	int	Entrada	Combustible requerido para llegar al destino	Entero positivo
`combustible_regreso`	int	Entrada	Combustible requerido para regresar	Entero positivo
`oxigeno_disponible`	int	Entrada	Oxígeno con que cuenta la nave	Entero positivo
`oxigeno_requerido`	int	Entrada	Oxígeno necesario para la misión	Entero positivo
`energia_disponible`	int	Entrada	Energía con que cuenta la nave	Entero positivo
`energia_requerida`	int	Entrada	Energía necesaria para la misión	Entero positivo
`provisiones_disponibles`	int	Entrada	Provisiones con que cuenta la nave	Entero positivo
`provisiones_requeridas`	int	Entrada	Provisiones necesarias para la misión	Entero positivo
`combustible_total`	int	Intermedia	Combustible total requerido (ida + regreso + reserva)	`combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE`

2. Diagrama de flujo



Ejercicio 4: Autorización inicial de lanzamiento

3. Código fuente (ejercicio4_autorizacion.py)

```
# ============================================================
# Archivo: ejercicio4_autorizacion.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV6
# Integrantes: Grace Hernandez Guevara
```

```
# Fecha: 14/10/2026     Versión: 1.0
# Descripción: Autoriza o rechaza el lanzamiento de una misión #
según el combustible, oxígeno, energía y provisiones.
# ============================================================

# --- Declaración de constantes --# Se declara al inicio para
evitar números mágicos en las fórmulas.
RESERVA_COMBUSTIBLE = 10

# --- Entrada de datos --combustible_disponible = int(input("Combustible
disponible: ")) combustible_ida = int(input("Combustible requerido para llegar
al destino: ")) combustible_regreso = int(input("Combustible requerido para
regresar: ")) oxigeno_disponible = int(input("Oxígeno disponible: "))
oxigeno_requerido = int(input("Oxígeno requerido: ")) energia_disponible =
int(input("Energía disponible: ")) energia_requerida = int(input("Energía
requerida: ")) provisiones_disponibles = int(input("Provisiones disponibles:
")) provisiones_requeridas = int(input("Provisiones requeridas: "))

# --- Procesamiento ---
# El combustible se compara contra el total (ida + regreso + reserva), no solo
# contra la ida: una nave que únicamente puede llegar quedaría varada.
combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE

# --- Decisión y salida de resultados --# Una sola condición compuesta con
"and": si falla cualquier recurso se rechaza.
# Se usa ">=" porque contar exactamente con lo requerido es suficiente. #
No se indica cuál recurso causó el rechazo, tal como lo pide la consigna.
if (combustible_disponible >= combustible_total          and
oxigeno_disponible >= oxigeno_requerido          and energia_disponible >=
energia_requerida          and provisiones_disponibles >=
provisiones_requeridas):
    print("Lanzamiento autorizado.") else:     print("Lanzamiento no
autorizado. La misión debe ser revisada.")
```

4. Casos de prueba

Entradas en el orden del programa: combustible disponible, ida, regreso; oxígeno disponible/requerido (O); energía disponible/requerida (E); provisiones disponibles/requeridas (P).

ID	Ejercicio	Escenario	Datos de entrada	Resultado esperado	Resultado obtenido	Estado
CP-15	Ejercicio 4	Todos los recursos suficientes	Comb. disp. = 150 Ida = 40, Regreso = 40 O = 100/65 E = 80/60 P = 50/33	Lanzamiento autorizado.	Igual al esperado	Exitoso
CP-16	Ejercicio 4	Todos exactamente iguales al mínimo (límite)	Comb. disp. = 90 Ida = 40, Regreso = 40 O = 65/65 E = 60/60 P = 33/33	Lanzamiento autorizado.	Igual al esperado	Exitoso
ID	**Ejercicio**	**Escenario**	**Datos de entrada**	**Resultado esperado**	**Resultado obtenido**	**Estado**
CP-17	Ejercicio 4	Combustible suficiente solo para llegar	Comb. disp. = 45 Ida = 40, Regreso = 40 O = 100/65 E = 80/60 P = 50/33	Lanzamiento no autorizado. La misión debe ser revisada.	Igual al esperado	Exitoso

CP-18	Ejercicio 4	Combustible para ida y regreso, pero sin la reserva	Comb. disp. = 85 Ida = 40, Regreso = 40 O = 100/65 E = 80/60 P = 50/33	Lanzamiento no autorizado. La misión debe ser revisada.	Igual al esperado	Exitoso
CP-19	Ejercicio 4	Oxígeno insuficiente (los demás suficientes)	Comb. disp. = 150 Ida = 40, Regreso = 40 O = 64/65 E = 80/60 P = 50/33	Lanzamiento no autorizado. La misión debe ser revisada.	Igual al esperado	Exitoso
CP-20	Ejercicio 4	Energía insuficiente (los demás suficientes)	Comb. disp. = 150 Ida = 40, Regreso = 40 O = 100/65 E = 59/60 P = 50/33	Lanzamiento no autorizado. La misión debe ser revisada.	Igual al esperado	Exitoso
CP-21	Ejercicio 4	Provisiones insuficientes (los demás suficientes)	Comb. disp. = 150 Ida = 40, Regreso = 40 O = 100/65 E = 80/60 P = 32/33	Lanzamiento no autorizado. La misión debe ser revisada.	Igual al esperado	Exitoso

Los resultados esperados se calcularon manualmente antes de ejecutar el programa.

5. Evidencia de ejecución

Captura CP-15 - Ejercicio 4

```
Terminal - CP-15 - Ejercicio 4
PS C:\mision-orbita\avance1> python ejercicio4_autorizacion.py
Combustible disponible: 150
Combustible requerido para llegar al destino: 40
Combustible requerido para regresar: 40
Oxígeno disponible: 100
Oxígeno requerido: 65
Energía disponible: 80
Energía requerida: 60
Provisiones disponibles: 50
Provisiones requeridas: 33
Lanzamiento autorizado.

PS C:\mision-orbita\avance1>
```

Captura CP-16 - Ejercicio 4

```
Terminal - CP-16 - Ejercicio 4
PS C:\mision-orbita\avance1> python ejercicio4_autorizacion.py
Combustible disponible: 90
Combustible requerido para llegar al destino: 40
Combustible requerido para regresar: 40
Oxígeno disponible: 65
Oxígeno requerido: 65
Energía disponible: 60
Energía requerida: 60
Provisiones disponibles: 33
Provisiones requeridas: 33
Lanzamiento autorizado.

PS C:\mision-orbita\avance1>
```

Captura CP-17 - Ejercicio 4

```
Terminal - CP-17 - Ejercicio 4
PS C:\mision-orbita\avance1> python ejercicio4_autorizacion.py
Combustible disponible: 45
Combustible requerido para llegar al destino: 40
Combustible requerido para regresar: 40
Oxígeno disponible: 100
Oxígeno requerido: 65
Energía disponible: 80
Energía requerida: 60
Provisiones disponibles: 50
Provisiones requeridas: 33
Lanzamiento no autorizado. La misión debe ser revisada.

PS C:\mision-orbita\avance1>
```

Captura CP-18 - Ejercicio 4

```
Terminal - CP-18 - Ejercicio 4
PS C:\mision-orbita\avance1> python ejercicio4_autorizacion.py
Combustible disponible: 85
Combustible requerido para llegar al destino: 40
Combustible requerido para regresar: 40
Oxígeno disponible: 100
Oxígeno requerido: 65
Energía disponible: 80
Energía requerida: 60
Provisiones disponibles: 50
Provisiones requeridas: 33
Lanzamiento no autorizado. La misión debe ser revisada.

PS C:\mision-orbita\avance1>
```

Captura CP-19 - Ejercicio 4

```
Terminal - CP-19 - Ejercicio 4
PS C:\mision-orbita\avance1> python ejercicio4_autorizacion.py
Combustible disponible: 150
Combustible requerido para llegar al destino: 40
Combustible requerido para regresar: 40
Oxígeno disponible: 64
Oxígeno requerido: 65
Energía disponible: 80
Energía requerida: 60
Provisiones disponibles: 50
Provisiones requeridas: 33
Lanzamiento no autorizado. La misión debe ser revisada.

PS C:\mision-orbita\avance1>
```

Captura CP-20 - Ejercicio 4

```
Terminal - CP-20 - Ejercicio 4
PS C:\mision-orbita\avance1> python ejercicio4_autorizacion.py
Combustible disponible: 150
Combustible requerido para llegar al destino: 40
Combustible requerido para regresar: 40
Oxígeno disponible: 100
Oxígeno requerido: 65
Energía disponible: 59
Energía requerida: 60
Provisiones disponibles: 50
Provisiones requeridas: 33
Lanzamiento no autorizado. La misión debe ser revisada.

PS C:\mision-orbita\avance1>
```

Captura CP-21 - Ejercicio 4

```
Terminal - CP-21 - Ejercicio 4
PS C:\mision-orbita\avance1> python ejercicio4_autorizacion.py
Combustible disponible: 150
Combustible requerido para llegar al destino: 40
Combustible requerido para regresar: 40
Oxígeno disponible: 100
Oxígeno requerido: 65
Energía disponible: 80
Energía requerida: 60
Provisiones disponibles: 32
Provisiones requeridas: 33
Lanzamiento no autorizado. La misión debe ser revisada.

PS C:\mision-orbita\avance1>
```